



# Analysis & Development of Algorithms

Anton Kovantsev

[https://t.me/pop\\_antoniij](https://t.me/pop_antoniij)

# Welcome & motivation.

## Why study algorithms at the master's level?

The gap between "using a library" and "designing for novel constraints."

**Real-world stakes:** latency budgets in HFT, energy constraints in edge computing, scalability in distributed systems.

**How AI Helps:** AI excels at optimization and exploration. It can rapidly generate boilerplate code, explore vast combinatorial search spaces (e.g., DeepMind's AlphaDev discovering novel, faster sorting routines), and automate empirical benchmarking. It is a powerful tool for refining algorithms within known boundaries.

**Why Humans are Irreplaceable:** The invention of a good algorithm requires deep human insight. AI extrapolates from existing data; it does not understand context. Humans are essential for:

- Problem Formulation: Translating messy, real-world constraints into rigorous mathematical models.
- Theoretical Guarantees: Proving correctness, establishing worst-case bounds, and preventing subtle logical flaws or hallucinations.
- Paradigm Shifts: Making the creative, non-obvious leaps required to invent entirely new algorithmic paradigms, rather than just optimizing old ones.



# Lectures content

**Lecture 1.** Introduction. Design of algorithms. Complexity analysis of algorithms. Significant operations. Experimental complexity analysis.

**Lecture 2.** Unconstrained nonlinear optimization. Direct methods: one-dimensional (Exhaustive search, Dichotomy, Golden section search) and multidimensional (Exhaustive search, Gauss method, Nelder-Mead method).

**Lecture 3.** Unconstrained nonlinear optimization. First- and second-order methods: Gradient Descent, Conjugate Gradient, Newton method, Quasi-Newton method, Levenberg-Marquardt algorithm.

**Lecture 4.** Unconstrained optimization algorithms. Stochastic and metaheuristic algorithms (Monte Carlo method, Simulated annealing, Evolutionary algorithms, Differential evolution, Particle swarm optimization).

**Lecture 5.** Introduction to graphs and basic algorithms on graphs. Graphs and their representations. Depth-first search and its applications. Breadth-first search and its applications.

**Lecture 6.** Graph algorithms. Path search algorithms on weighted graphs. Dijkstra's algorithm. A\* algorithm. Bellman-Ford algorithm

**Lecture 7.** Graph algorithms. Tools for network analysis.

**Lecture 8.** Analysis of modern algorithms.

# Practical tasks

- 1) Experimental time complexity analysis.
- 2) Unconstraint nonlinear optimization. Direct algorithms.
- 3) Unconstraint nonlinear optimization algorithms. First- and second-order methods.
- 4) Unconstraint optimization algorithms. Stochastic and metaheuristic algorithms.
- 5) Graphs and their representations. Depth-first search and its applications. Breadth-first search and its applications.
- 6) Path search algorithms on weighted graphs. Dijkstra's algorithm. A\* algorithm. Bellman-Ford algorithm.
- 7) Graph algorithms. Tools for network analysis.
- 8) Analysis of modern algorithms.

## **References:** Algorithms and data structures

1. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest; Clifford Stein. Introduction to Algorithms. 3rd edition. MIT Press, 2009.
2. Ananiy Levitin. Introduction to the Design and Analysis of Algorithms. Addison Wesley, 2011.

Other textbooks, lecture notes and YouTube tutorials

The course is for refreshing some known information and making the knowledge level of students from different bachelor programs approximately equal.

# Why do we study algorithms?

## **Problem Solving**

Solving problems that might otherwise be intractable. A structured approach to addressing complex issues, making them essential for tackling a wide range of problems in different fields.

## **Broad Applications**

Internet, biology, computer graphics, security, social networks, particle collision simulations, and quantum computing.

## **Programming Proficiency**

Good programmers focus on data structures and their relationships, which are central to algorithm design.

## **Efficiency and Productivity**

Help in optimizing processes, making them more efficient and productive.

## **Intellectual Stimulation**

Offer a form of intellectual stimulation, often described as the "poetry of computation."

## **Common Language and Understanding**

Common language for understanding and describing computational processes. Modelling and analysing complex systems, replacing traditional mathematical models with computational ones.

## **Evaluation and Comparison**

Analyzing the efficiency, resource usage, and scalability of various algorithms, which is essential for identifying the best approach to a given problem.

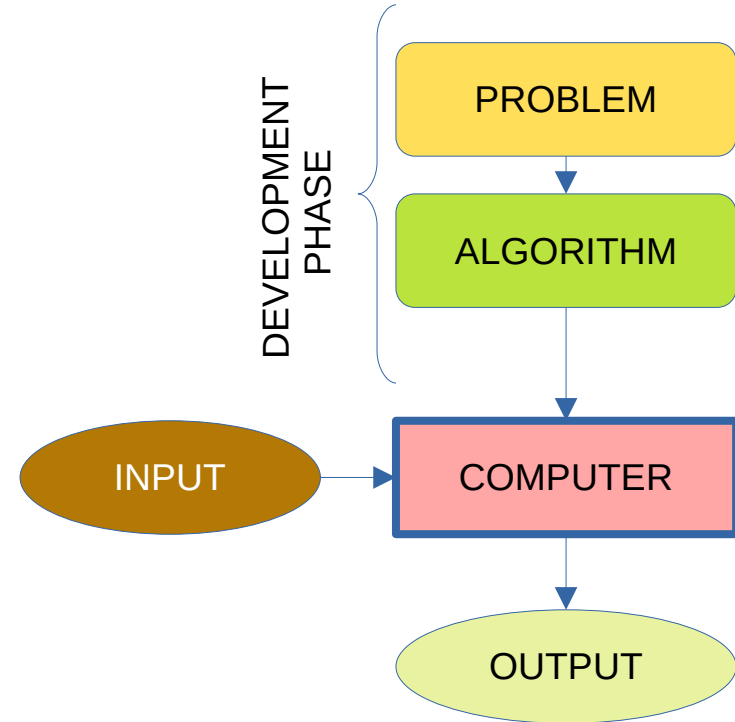
## **Automation and Decision Making**

Automation and decision-making in various industries, such as AI, medical diagnosis, and data analysis.

# Definition

**An algorithm** is a sequence of clear instructions for solving a problem, i.e., for obtaining a required output for any acceptable input in a finite amount of time.

- The class of inputs for which an algorithm works has to be specified carefully.
- The same algorithm can be represented in several different ways.
- There may exist several algorithms for solving the same problem.
- Algorithms for the same problem can be based on very different ideas and
- can solve the problem with dramatically different speed.



# Algorithm development process

## Steps in development of Algorithms

- 1) Problem definition
- 2) Development of a model
- 3) Specification of Algorithm
- 4) Designing an Algorithm
- 5) Checking the correctness of Algorithm
- 6) Analysis of Algorithm
- 7) Implementation of Algorithm
- 8) Program testing
- 9) Documentation Preparation

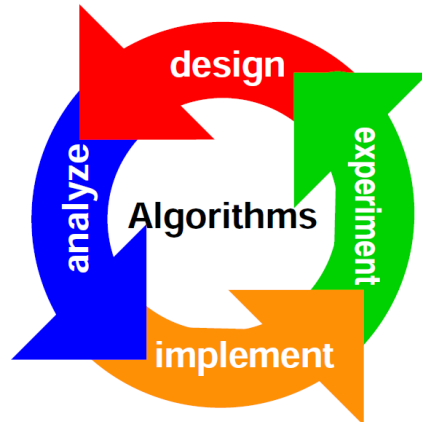
## Standard Methods in solving a Problem

A problem can either be solve using any or both of these methods:

1. Top Down Method
2. Bottom Top Method

The **Top Down Method** starts to solve a problem at the most general level and works toward details or specifics. It is also an approach to a problem that begins at the highest conceptual level and works down to the details.

The **Bottom Top Method** is an approach to a problem that begins with details and works up to the highest conceptual level.



# Qualities of a Good Algorithm

- 1. Simplicity/Effectiveness:** The algorithm must be simple and not complex to follow in solving a particular problem. The operations performed by an algorithm should be simple so that they can be done exactly and in a finite length of time by some using pencil and paper.
- 2. Clarity/Definiteness:** The algorithm should be clear and easy to read and understand. In other words the algorithm must be precisely defined and unambiguous.
- 3. Thoroughness:** A thorough algorithm is one that considers all the possible cases and situations that may arise in that particular problem. This can be achieved by making use of variables in storing input.
- 4. Correctness:** The algorithm being developed must be correct. This means that the algorithm must produce the correct result/solution of the problem. However if the result is not correct, then the algorithm should be modified. If however the algorithm could not produce the correct result/solution then we will say that the algorithm did not solve the problem or that the algorithm is wrong.



# Qualities of a Good Algorithm

**5. Efficiency:** This is the most important of all. This means that the algorithm should be the best in time consumed to solve the particular problem. The amount of computer resources that requires to solve the problem should be considered when developing an algorithm. An algorithm that requires small memory space and less time to reach the solution of a particular problem is better considered/Taking.

**6. Finiteness:** The actual steps executed when an algorithm is run should be of a reasonable size. This means that a execution time of algorithm should be of reasonable length. The algorithm must be finite. If an old computer could not boot before the user dies, then the computer is useless.

**7. Input:** An algorithm may have some input(s), from a specified device. This input(s) may be from an external device (like Hard disc, Flash drive) or within the computer (e.g. keyboard, RAM). The commonest input in an algorithm includes Numbers – the integers numbers, real numbers, character etc.

**8. Output:** An algorithm must have output(s). The output is always but not limited to the result of the algorithm. The output may be displayed in the visual displaying unit (VDU) or stored in the computer memory for other programme/algorithm to use.

And that is it! The algorithm that posses all these qualities should be chosen from millions of equivalent algorithms that solve the same problem.

# Algorithm development process

## Understanding the problem

## Determining the capabilities of a computing device

Sequential algorithms

Parallel algorithms

Learning algorithms without being tied to the parameters of a specific computer system

## Choosing between exact or approximate method for solving the problem

Exact method

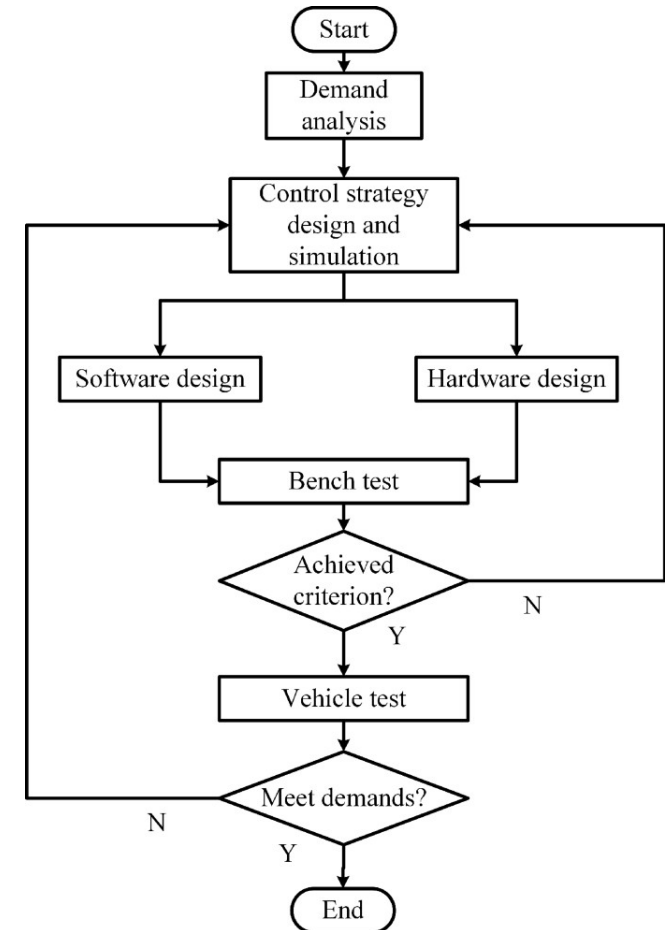
Approximation method

## Choosing the right data structures

Many algorithms require very specific data structures

## Algorithm Design Techniques

A design technique is a versatile approach applied to algorithmic solutions to a wide range of problems



# Algorithm development process

## Algorithm Representation Methods

- Natural language
- Pseudocode: a mixture of one of the natural languages (input / output, comments) and constructs specific to the programming language (for, if and while)
- Flowcharts: drawings consisting of a sequence of geometric shapes connected by arrows, with the help of which each step of the algorithm execution is described (inconvenient for large algorithms)

## Algorithm Correctness Evaluation

Prove that the chosen algorithm produces the required result for any correct values of the input data in a limited period of time (mathematical induction is a universal method)

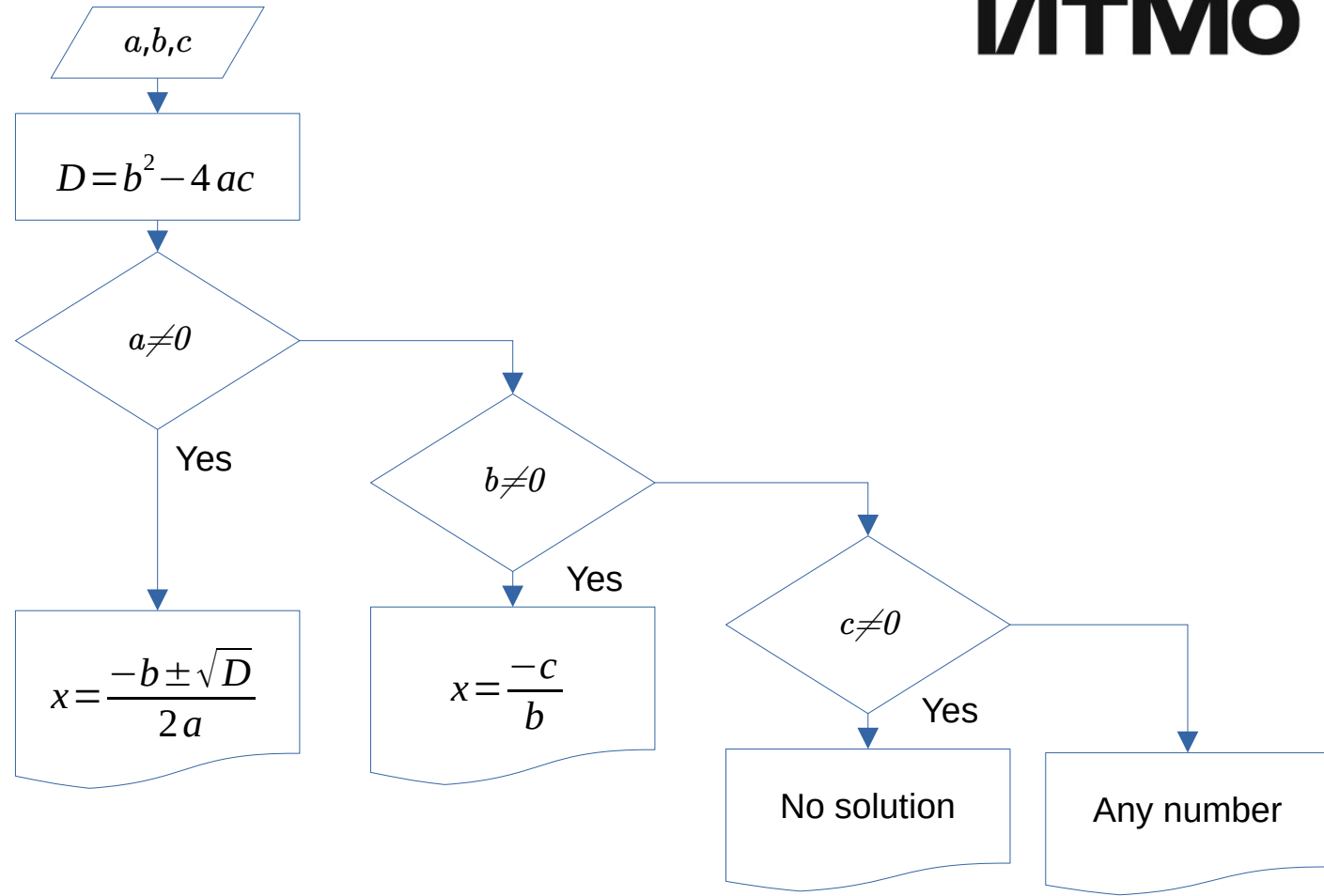
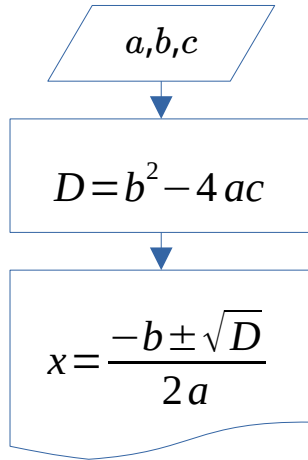
## Algorithm Analysis

- Time efficiency, or time complexity: running time function
- Space efficiency, or space complexity: the amount of RAM required for the algorithm to work
- Simplicity
- Generality, or universality
- Optimality

## Algorithm Encoding, debugging and testing

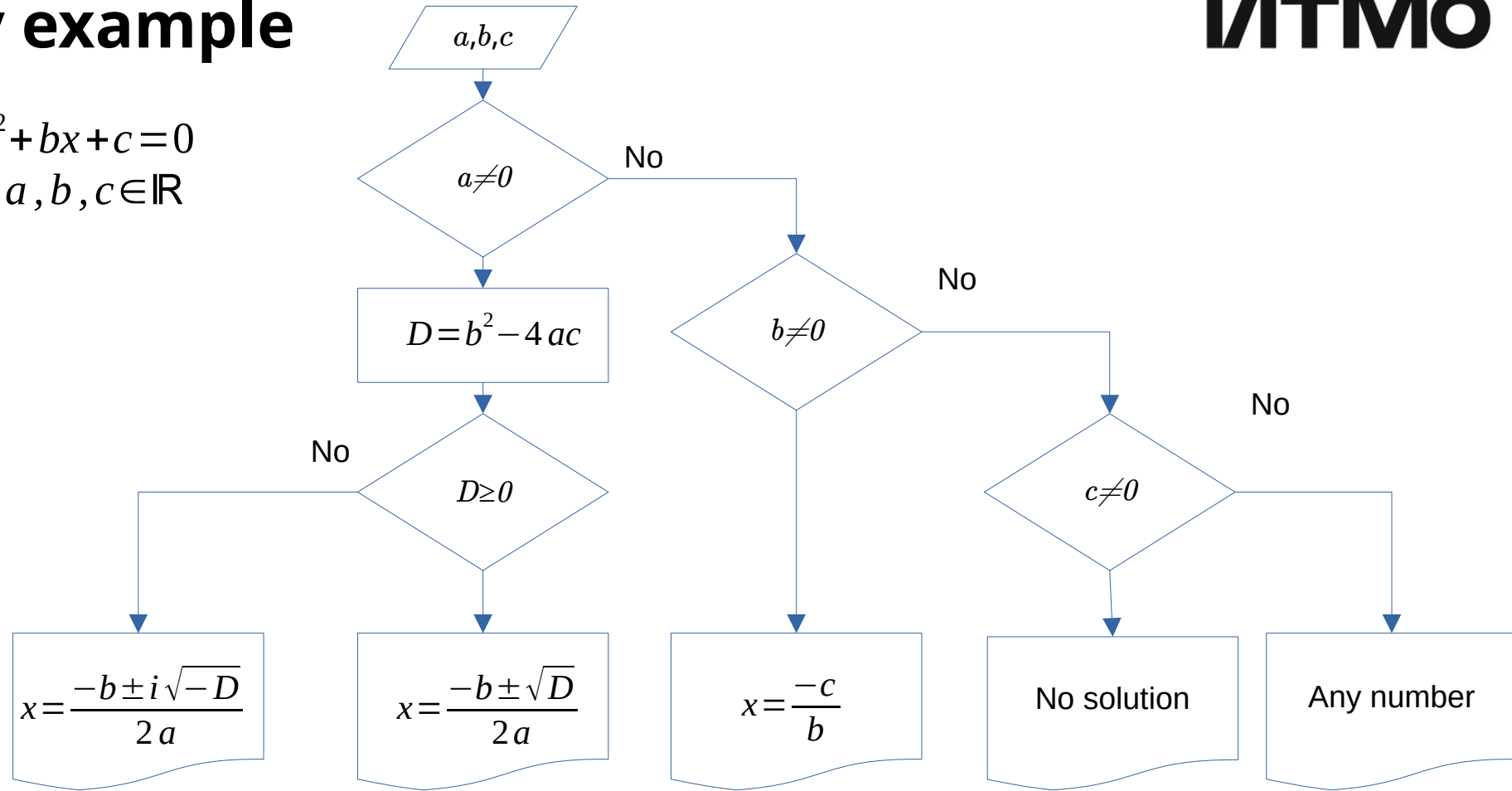
# Toy example

$$ax^2+bx+c=0$$



# Toy example

$$ax^2+bx+c=0$$
$$\forall a,b,c \in \mathbb{R}$$

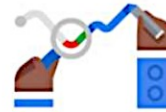


# Important Problem Types

- **Sorting:** to rearrange the items of a given list in some order (e.g. using a key)
- **Searching:** to find a given value, called a search key, in a given (multi)set
- **String processing:** to process a string that is a sequence of characters from an alphabet
- **Graph problems:** to analyse a graph, i.e. a collection of points called vertices, some of which are connected by line segments called edges, informally
- **Combinatorial problems:** to find a combinatorial object – such as a permutation, a combination, or a subset – that satisfies certain constraints
- **Geometric problems:** to explore object geometry (topology as well)
- **Numerical problems:** to solve equations and systems of equations, compute definite integrals, evaluate functions, etc.

For Data Analysis

1. Making predictions



2. Categorizing things



3. Spotting something unusual



4. Identifying themes



5. Discovering connections



6. Finding patterns



# Algorithm Performance

The term “**analysis of algorithms**” usually means an investigation of algorithm’s efficiency with respect to two resources: running time and memory space (aka computational complexity).

## Time

- Instructions take time.
- How fast does the algorithm perform?
- What affects its runtime?

## Space

- Data structures take space
- What kind of data structures can be used?
- How does choice of data structure affect the runtime?

If the problem size is always very small, we can probably ignore the algorithm’s efficiency.

We have to find the trade-off between the algorithm’s time requirement and its memory requirements.

How to estimate the time required for an algorithm?

**Naive Approach:** run an algorithm for different size of input data and analyse the results.

Analyse algorithms **independently** of specific implementations, computers, or data.

- First, we start to count the number of significant operations in a particular solution to assess its efficiency.
- Then, we will express the efficiency of algorithms using growth functions.

# Analysis of Algorithms (Worst Case)

A running time function,  $T(n)$ , yields the time required to execute the algorithm of a problem of size  $n$ .

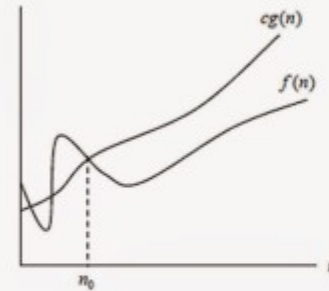
- $T(n)$  may contain unspecified constants.
- We cannot determine this function exactly.
- Example:  $T(n) = an^2 + bn + c$ , where  $a, b, c$  are unspecified constants.

We can compare the efficiency of two algorithms by comparing their growth rates. The lower the growth rate, the faster the algorithm, at least for large values of  $n$ .

For example,  $T(n) = an^2 + bn + c$  has growth rate  $O(n^2)$

The goal of the algorithm designer: an algorithm with as low a growth rate of the running function,  $T(n)$ , of that algorithm as possible.

## The (Big) O Notation



$O(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq f(n) \leq cg(n) \text{ for all } n \geq n_0\}.$

$g(n)$  is an asymptotic upper bound for  $f(n)$ .

### Examples:

$$n^2 = O(n^2)$$

$$n^2 + n = O(n^2)$$

$$n^2 + 1000n = O(n^2)$$

$$5230n^2 + 1000n = O(n^2)$$

$$n = O(n^2)$$

$$\frac{n}{1200} = O(n^2)$$

$$n^{1.99999} = O(n^2)$$

$$\frac{n^2}{\log n} = O(n^2)$$

**Note:** Since changing the base of a log only changes the function by a constant factor, we usually don't worry about log bases in asymptotic notation.



# Why do we study algorithms?

## Loops

- The running time of a loop is at most the running time of the statements inside of that loop times the number of iterations.

## Nested Loops

- Running time of a nested loop containing a statement in the inner most loop is the running time of statement multiplied by the product of the size of all loops.

## Consecutive Statements

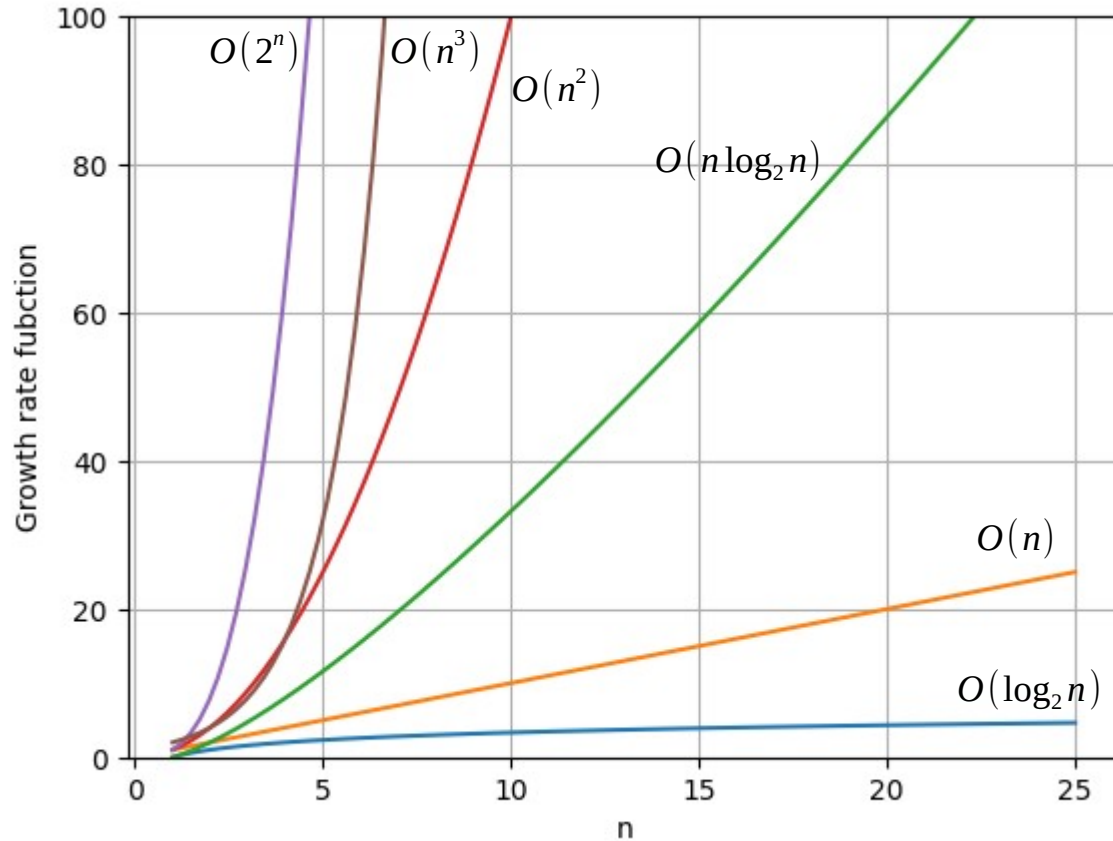
- Just add the running times of those consecutive statements.

## If/Else

- Never more than the running time of the test plus the larger of running times of the options.

| order of growth | name        | typical code framework                                                                                                              | description       | example           |
|-----------------|-------------|-------------------------------------------------------------------------------------------------------------------------------------|-------------------|-------------------|
| 1               | constant    | <pre>a = b + c;</pre>                                                                                                               | statement         | add two numbers   |
| $\log N$        | logarithmic | <pre>while (N &gt; 1) {   N = N / 2;  ... }</pre>                                                                                   | divide in half    | binary search     |
| N               | linear      | <pre>for (int i = 0; i &lt; N; i++) {   ... }</pre>                                                                                 | loop              | find the maximum  |
| $N^2$           | quadratic   | <pre>for (int i = 0; i &lt; N; i++)   for (int j = 0; j &lt; N; j++)   {     ...   }</pre>                                          | double loop       | check all pairs   |
| $N^3$           | cubic       | <pre>for (int i = 0; i &lt; N; i++)   for (int j = 0; j &lt; N; j++)     for (int k = 0; k &lt; N; k++)     {       ...     }</pre> | triple loop       | check all triples |
| $2^N$           | exponential | Combinatorial algorithms                                                                                                            | exhaustive search | check all subsets |

# A Comparison of Growth-Rate Functions

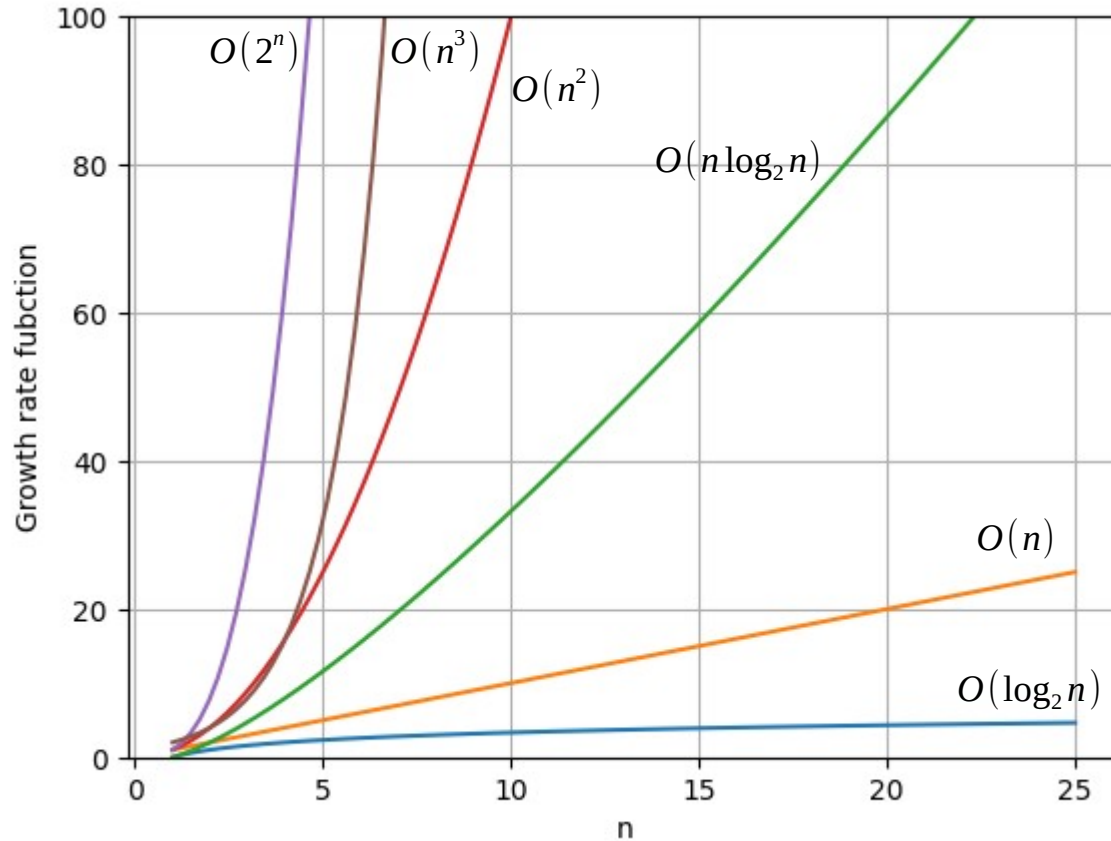


**Example:** Simple If-Statement  
The input is  $n$

```
if (n < 0): #Cost c1 = 1
    av = -n; #Cost c2 = 1
else:
    av = n; #Cost c3 = 1
```

$Total\ Cost = c1 + \max(c2, c3) = O(1)$

# A Comparison of Growth-Rate Functions

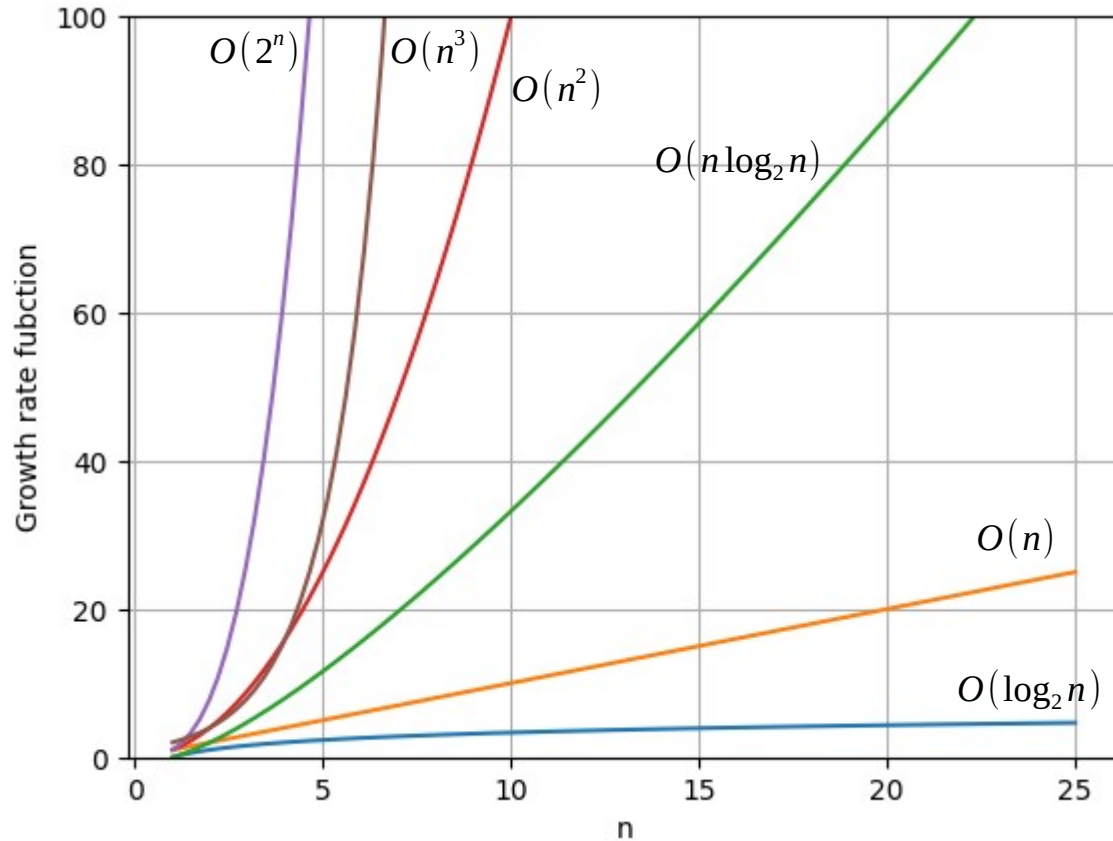


**Example:** Simple loop  
The input is  $n$

```
av=0                                #Cost c0 = 1
for i in range(n):                  #Cost c1 = N
    if (i % 3):                      #Cost c2 = 1
        av += -i;                   #Cost c3 = 1
    else:
        av += i;                    #Cost c4 = 1
```

$$\text{Total Cost} = \max(c0, c1 * (c2 + \max(c3, c4))) \\ = O(N)$$

# A Comparison of Growth-Rate Functions



**Example:** Not very simple loop  
The input is n

```
av = 0                                #Cost c0 = 1
while n < 1:                            #Cost c1 = log2n
    n = n/2
    if (n % 5): #Cost c2 = 1
        av += -n; #Cost c3 = 1
    else:
        av += n; #Cost c4 = 1
```

$$\text{Total Cost} = \max(c_0, c_1 * (c_2 + \max(c_3, c_4))) \\ = O(\log_2 n)$$

# The Execution Time of Algorithms

*Example: Nested Loop*

*The input is n*

|                  | <u>Cost</u> | <u>Times</u> |
|------------------|-------------|--------------|
| i=1;             | c1          | 1            |
| sum = 0;         | c2          | 1            |
| while (i <= n) { | c3          | n+1          |
| j=1;             | c4          | n            |
| while (j <= n) { | c5          | n*(n+1)      |
| sum = sum + i;   | c6          | n*n          |
| j = j + 1;       | c7          | n*n          |
| }                |             |              |
| i = i + 1;       | c8          | n            |
| }                |             |              |

$$\text{Total Cost} = c1 + c2 + (n+1)*c3 + n*c4 + n*(n+1)*c5 + n*n*c6 + n*n*c7 + n*c8 = O(n^2)$$

# Basic Data Structures

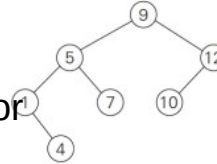
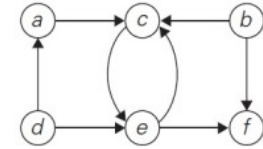
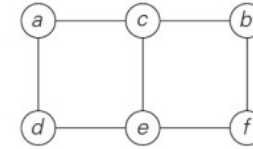
A **data structure** is a particular scheme of organizing related data items.

The nature of the data items is dictated by the problem at hand; they can range from elementary data types (e.g., integers or characters) to data structures (e.g., a one-dimensional array of one-dimensional arrays is often used for implementing matrices).

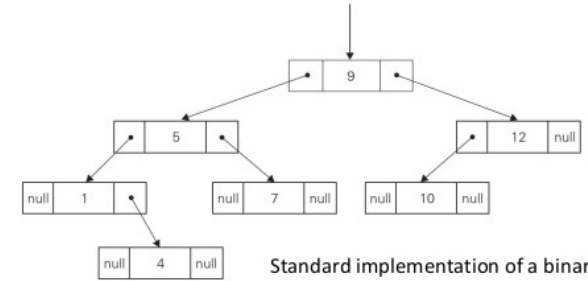
There are a few data structures that have proved to be particularly important for computer algorithms.

An abstract collection of objects with several operations that can be performed on them is called an abstract data type.

List, stack, queue, priority queue, and dictionary are important examples of abstract data types.



Binary search tree



Standard implementation of a binary tree

A binary tree is usually implemented for computing purposes by a collection of nodes corresponding to vertices of the tree.

Each node contains some information associated with the vertex (its name or some value assigned to it) and two pointers to the nodes representing the left child and right child of the vertex, respectively.

# Basic Data Structures

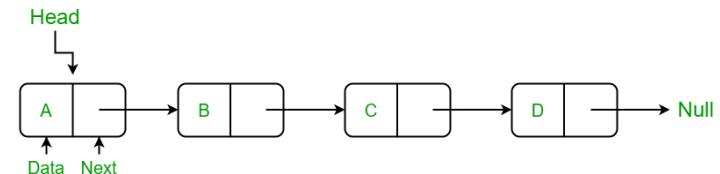
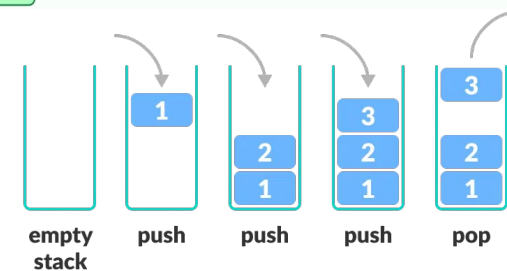
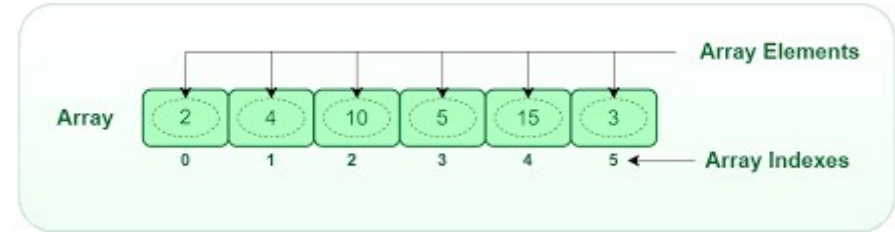
A **data structure** is a particular scheme of organizing related data items.

The nature of the data items is dictated by the problem at hand; they can range from elementary data types (e.g., integers or characters) to data structures (e.g., a one-dimensional array of one-dimensional arrays is often used for implementing matrices).

There are a few data structures that have proved to be particularly important for computer algorithms.

An abstract collection of objects with several operations that can be performed on them is called an abstract data type.

List, stack, queue, priority queue, and dictionary are important examples of abstract data types.

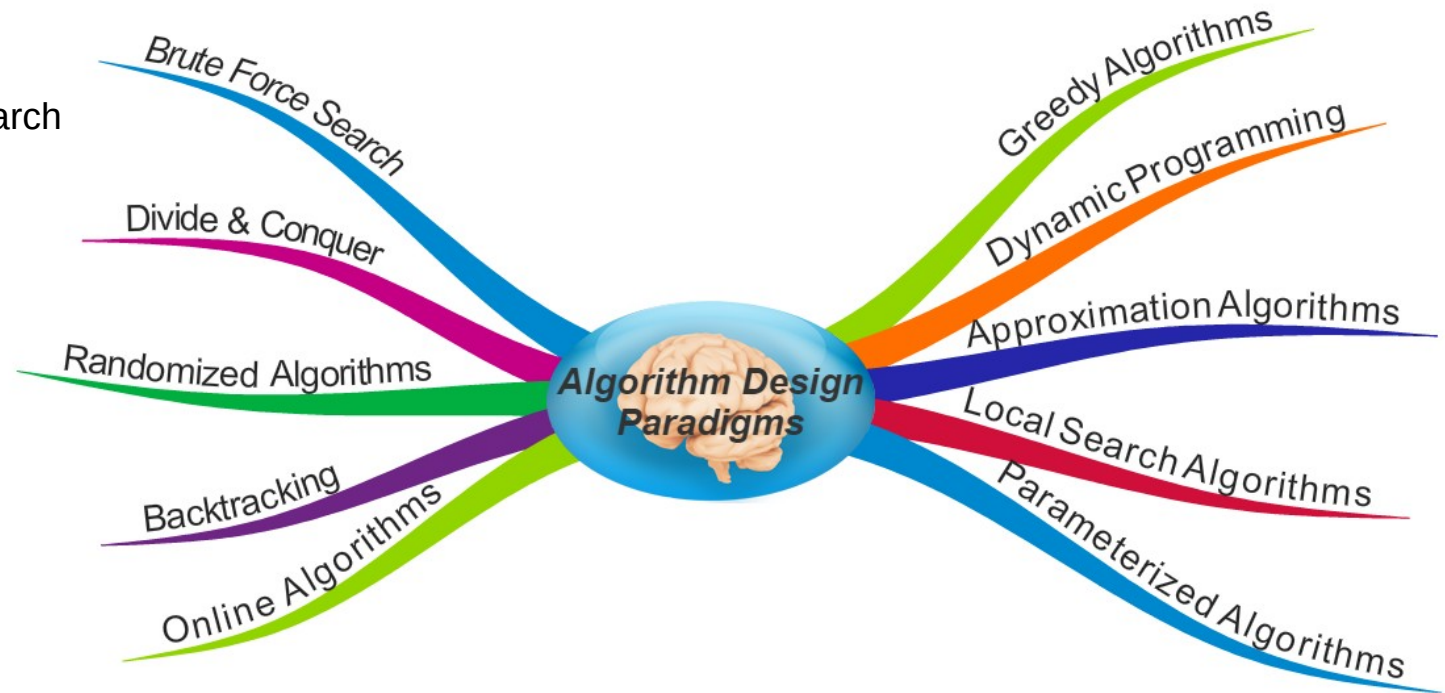


# Algorithm design techniques

**Algorithm design** techniques (or “strategies” or “paradigms”) are general approaches to solving problems algorithmically, applicable to a variety of problems from different areas of computing.

## Basic techniques:

- Brute-force or exhaustive search
- Divide and Conquer
- Greedy Algorithms
- Dynamic Programming
- Backtracking
- etc.







АНТОН КОВАНЦЕВ  
[pop.antonij@gmail.com](mailto:pop.antonij@gmail.com)  
[https://t.me/pop\\_antonij](https://t.me/pop_antonij)